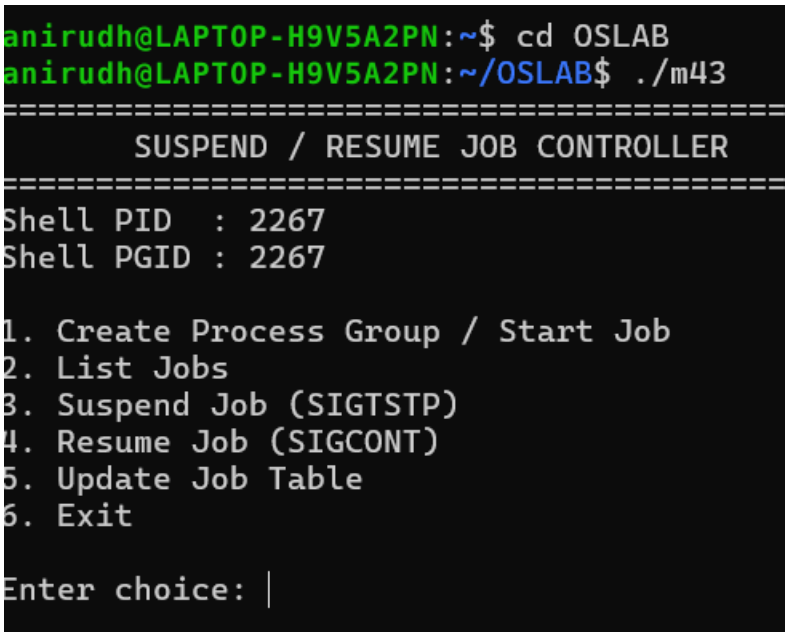

anirudh@LAPTOP-H9V5A2PN:~$ cd OSLAB
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m43
SUSPEND / RESUME JOB CONTROLLER
Shell PID : 2267
Shell PGID : 2267
1. Create Process Group / Start Job
2. List Jobs
3. Suspend Job (SIGTSTP)
4. Resume Job (SIGCONT)
5. Update Job Table
6. Exit
Enter choice:

```c
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 10

typedef enum
{
    JOB_RUNNING,
    JOB_STOPPED,
    JOB_DONE
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    JobState state;
    char command[100];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

volatile sig_atomic_t foreground_pgid = -1;

const char *state_name(JobState state)
{
    if (state == JOB_RUNNING)
        return "Running";

    if (state == JOB_STOPPED)
        return "Stopped";

    return "Done";
}

/* Capture SIGTSTP from terminal */
void sigtstp_handler(int sig)
{
    if (foreground_pgid > 0)
    {
        printf("\nShell: SIGTSTP received.\n");
        printf("Shell: Forwarding SIGTSTP to PGID %d\n",
               foreground_pgid);

        kill(-foreground_pgid, sig);
    }
}

/* Add process to job table */
void add_job(pid_t pid, pid_t pgid)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pgid;
    jobs[job_count].state = JOB_RUNNING;

    snprintf(
        jobs[job_count].command,
        sizeof(jobs[job_count].command),
```

```
        printf("\nShell: SIGTSTP received.\n");
        printf("Shell: Forwarding SIGTSTP to PGID %d\n",
               foreground_pgid);

        kill(-foreground_pgid, sig);
    }
}

/* Add process to job table */
void add_job(pid_t pid, pid_t pgid)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pgid;
    jobs[job_count].state = JOB_RUNNING;

    snprintf(
        jobs[job_count].command,
        sizeof(jobs[job_count].command),
        "foreground-process"
    );

    job_count++;
}

/* Update job state using waitpid */
void update_job_table()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == JOB_DONE)
            continue;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED | WCONTINUED
        );

        if (result == jobs[i].pid)
        {
            if (WIFSTOPPED(status))
            {
                jobs[i].state = JOB_STOPPED;

                printf(
                    "\nJob [%d] STOPPED by signal %d\n",
                    jobs[i].job_id,
                    WSTOPSIG(status)
                );
            }
            else if (WIFCONTINUED(status))
            {
                jobs[i].state = JOB_RUNNING;

                printf(
                    "\nJob [%d] RESUMED\n",
                    jobs[i].job_id
                );
```

```
                jobs[i].state = JOB_DONE;

                printf(
                    "\nJob [%d] completed with status %d\n",
                    jobs[i].job_id,
                    WEXITSTATUS(status)
                );
            }
            else if (WIFSIGNALED(status))
            {
                jobs[i].state = JOB_DONE;

                printf(
                    "\nJob [%d] terminated by signal %d\n",
                    jobs[i].job_id,
                    WTERMSIG(status)
                );
            }
        }
    }
}

/* Display job table */
void list_jobs()
{
    update_job_table();

    printf("\n============================================\n");
    printf("                JOB TABLE\n");
    printf("============================================\n");

    printf(
        "%-8s %-10s %-10s %-12s %s\n",
        "Job",
        "PID",
        "PGID",
        "State",
        "Command"
    );

    printf(
        "------------------------------------------------------------\n"
    );

    for (int i = 0; i < job_count; i++)
    {
        printf(
            "%-8d %-10d %-10d %-12s %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].pgid,
            state_name(jobs[i].state),
            jobs[i].command
        );
    }
}

/* Resume stopped job */
void resume_job(int job_id)
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == job_id)
        {
            if (jobs[i].state != JOB_STOPPED)
            {
                printf(
                    "Job [%d] is not stopped.\n",
                    job_id
                );
                return;
            }
```

```c
void resume_job(int job_id)
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == job_id)
        {
            if (jobs[i].state != JOB_STOPPED)
            {
                printf(
                    "Job [%d] is not stopped.\n",
                    job_id
                );
                return;
            }

            printf(
                "\nSending SIGCONT to process group %d...\n",
                jobs[i].pgid
            );

            if (kill(-jobs[i].pgid, SIGCONT) == -1)
            {
                perror("kill SIGCONT");
                return;
            }

            jobs[i].state = JOB_RUNNING;

            printf(
                "Job [%d] resumed successfully.\n",
                job_id
            );

            return;
        }
    }

    printf("Job [%d] not found.\n", job_id);
}

/* Create a process and process group */
void start_job()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        /*
         * Create a new process group.
         * Child becomes process group leader.
         */
        setpgid(0, 0);

        /*
         * Child receives default terminal signals.
         */
        signal(SIGTSTP, SIG_DFL);
        signal(SIGINT, SIG_DFL);

        printf("\n=========================================\n");
        printf("            CHILD PROCESS STARTED\n");
        printf("=========================================\n");

        printf("PID  : %d\n", getpid());
        printf("PGID : %d\n", getpgrp());
```

```
    }

    /*
     * Parent also assigns child to its process group.
     */
    setpgid(pid, pid);

    add_job(pid, pid);

    printf(
        "\nJob [%d] created.\n",
        job_count
    );

    printf("PID  : %d\n", pid);
    printf("PGID : %d\n", pid);
}

/* Stop a job manually */
void stop_job(int job_id)
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == job_id)
        {
            printf(
                "\nSending SIGTSTP to Job [%d]...\n",
                job_id
            );

            if (kill(-jobs[i].pgid, SIGTSTP) == -1)
            {
                perror("kill SIGTSTP");
                return;
            }

            jobs[i].state = JOB_STOPPED;

            printf(
                "Job [%d] stopped.\n",
                job_id
            );

            return;
        }
    }

    printf("Job [%d] not found.\n", job_id);
}

int main()
{
    int choice;
    int job_id;

    /*
     * Shell ignores SIGTTIN and SIGTTOU.
     */
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTTOU, SIG_IGN);

    /*
     * Register SIGTSTP handler.
     */
    struct sigaction action;

    action.sa_handler = sigtstp_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = SA_RESTART;

    if (sigaction(SIGTSTP, &action, NULL) == -1)
    {
```

```
printf("Shell PID  : %d\n", getpid());
printf("Shell PGID : %d\n", getpgrp());

while (1)
{
    printf("\n");
    printf("1. Create Process Group / Start Job\n");
    printf("2. List Jobs\n");
    printf("3. Suspend Job (SIGTSTP)\n");
    printf("4. Resume Job (SIGCONT)\n");
    printf("5. Update Job Table\n");
    printf("6. Exit\n");

    printf("\nEnter choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            start_job();
            break;

        case 2:
            list_jobs();
            break;

        case 3:
            printf("Enter Job ID: ");
            scanf("%d", &job_id);

            stop_job(job_id);
            break;

        case 4:
            printf("Enter Job ID: ");
            scanf("%d", &job_id);

            resume_job(job_id);
            break;

        case 5:
            update_job_table();
            list_jobs();
            break;

        case 6:
            for (int i = 0; i < job_count; i++)
            {
                if (jobs[i].state != JOB_DONE)
                {
                    kill(-jobs[i].pgid, SIGTERM);
                }
            }

            printf("\nAll jobs terminated.\n");
            printf("Shell exiting.\n");

            return 0;

        default:
            printf("Invalid choice.\n");
    }
}

return 0;
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m44.c -o m44
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m44
==========================================
       PROCESS GROUP & TERMINAL CONTROL
==========================================
Shell PID  : 2430
Shell PGID : 2430
Terminal PGID: 2430

==========================================
          PROCESS GROUP CREATED
==========================================
Child PID  : 2431
Child PGID : 2431
Terminal control transferred to PGID 2431

==========================================
              CHILD PROCESS
==========================================
Child PID  : 2431
Child PGID : 2431

Child running...
Press Ctrl+C to send SIGINT to this group.
Press Ctrl+Z to stop this group.
Foreground process group 2431 working...
Foreground process group 2431 working...
```

```
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m44.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m44.c -o m44
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m44
==========================================
       PROCESS GROUP & TERMINAL CONTROL
==========================================
Shell PID  : 2430
Shell PGID : 2430
Terminal PGID: 2430

==========================================
          PROCESS GROUP CREATED
==========================================
Child PID  : 2431
Child PGID : 2431
Terminal control transferred to PGID 2431

==========================================
              CHILD PROCESS
==========================================
Child PID  : 2431
Child PGID : 2431

Child running...
Press Ctrl+C to send SIGINT to this group.
Press Ctrl+Z to stop this group.
Foreground process group 2431 working...
Foreground process group 2431 working...
Foreground process group 2431 working...

^C
Child terminated by signal 2
Terminal ownership restored to shell PGID 2430

==========================================
              TEST COMPLETED
==========================================
Process group coordination verified.
Terminal ownership restored.
Signal handling completed.
```

```
anirudh@LAPTOP-H9V5AZPN:~/OSLAB$ cat m44.c
/* process_group_terminal_control.c
 *
 * Create Process Groups
 * Assign Group IDs
 * Transfer Terminal Control
 * Restore Ownership
 * Coordinate Signals
 * Validate Behavior
 *
 * Compile:
 * gcc process_group_terminal_control.c -o process_group_terminal_control
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

pid_t shell_pgid;
pid_t child_pgid;

void ignore_shell_signals()
{
    signal(SIGINT, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);
    signal(SIGQUIT, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTTOU, SIG_IGN);
}

void child_signal_defaults()
{
    signal(SIGINT, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);
    signal(SIGQUIT, SIG_DFL);
    signal(SIGTTIN, SIG_DFL);
    signal(SIGTTOU, SIG_DFL);
}

void transfer_terminal(pid_t pgid)
{
    if (tcsetpgrp(STDIN_FILENO, pgid) == -1)
    {
        perror("tcsetpgrp");
    }
    else
    {
        printf(
            "Terminal control transferred to PGID %d\n",
            pgid
        );
    }
}

void restore_terminal()
{
    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    {
        perror("tcsetpgrp restore");
    }
    else
    {
        printf(
            "Terminal ownership restored to shell PGID %d\n",
            shell_pgid
        );
```

```
    {
        perror("tcsetpgrp restore");
    }
    else
    {
        printf(
            "Terminal ownership restored to shell PGID %d\n",
            shell_pgid
        );
    }
}

int main()
{
    pid_t pid;
    int status;

    shell_pgid = getpgrp();

    printf("============================================\n");
    printf("       PROCESS GROUP & TERMINAL CONTROL\n");
    printf("============================================\n");

    printf("Shell PID  : %d\n", getpid());
    printf("Shell PGID : %d\n", shell_pgid);
    printf("Terminal PGID: %d\n", tcgetpgrp(STDIN_FILENO));

    /*
     * Protect shell from terminal-generated signals.
     */
    ignore_shell_signals();

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        /*
         * Create a new process group.
         */
        setpgid(0, 0);

        child_pgid = getpgrp();

        /*
         * Restore default signal behavior.
         */
        child_signal_defaults();

        printf("\n===========================================\n");
        printf("              CHILD PROCESS\n");
        printf("===========================================\n");

        printf("Child PID  : %d\n", getpid());
        printf("Child PGID : %d\n", child_pgid);

        printf("\nChild running...\n");
        printf("Press Ctrl+C to send SIGINT to this group.\n");
        printf("Press Ctrl+Z to stop this group.\n");

        while (1)
        {
            printf(
                "Foreground process group %d working...\n",
                getpgrp()
            );
```

```
    printf("\n==========================================\n");
    printf("          PROCESS GROUP CREATED\n");
    printf("==========================================\n");

    printf("Child PID  : %d\n", pid);
    printf("Child PGID : %d\n", child_pgid);

    /*
     * Give terminal ownership to child group.
     */
    transfer_terminal(child_pgid);

    /*
     * Wait for child to stop or terminate.
     */
    while (1)
    {
        pid_t result = waitpid(
            pid,
            &status,
            WUNTRACED
        );

        if (result == -1)
        {
            if (errno == EINTR)
                continue;

            perror("waitpid");
            break;
        }

        if (WIFSTOPPED(status))
        {
            printf(
                "\nChild stopped by signal %d\n",
                WSTOPSIG(status)
            );

            printf("Sending SIGCONT to child process group...\n");

            if (kill(-child_pgid, SIGCONT) == -1)
            {
                perror("SIGCONT");
            }

            sleep(1);

            printf("Stopping child group again for demonstration...\n");

            if (kill(-child_pgid, SIGTSTP) == -1)
            {
                perror("SIGTSTP");
            }

            continue;
        }

        if (WIFEXITED(status))
        {
            printf(
                "\nChild exited with status %d\n",
                WEXITSTATUS(status)
            );

            break;
        }

        if (WIFSIGNALED(status))
        {
            printf(
```

```
        }

        if (WIFSIGNALED(status))
        {
            printf(
                "\nChild terminated by signal %d\n",
                WTERMSIG(status)
            );

            break;
        }
    }

    /*
     * Restore terminal ownership to shell.
     */
    restore_terminal();

    printf("\n==========================================\n");
    printf("              TEST COMPLETED\n");
    printf("==========================================\n");

    printf("Process group coordination verified.\n");
    printf("Terminal ownership restored.\n");
    printf("Signal handling completed.\n");

    return 0;
}
anirudh@LAPTOP-H0V5A2RN:~/OSLAB$
```